2024 IEEE Symposium on Security and Privacy (SP)

SrcMarker: Dual-Channel Source Code Watermarking via Scalable Code Transformations

Fengrui Liu*

* Nanjing University of Posts and Telecommunications

Email:liuferry@163.com

Abstract--The widespread adoption of open-source communities and the rapid development of large language models have raised new ethical and security challenges regarding source code distribution. These challenges include unauthorized usage, copyright infringement, and potential misuse of source code. Traditional watermarking solutions, either operating on compiled binaries or relying on natural language text approaches, face limitations in maintaining code functionality and readability. In this work, we present SrcMarker, a novel dual-channel source code watermarking framework that leverages both the natural and formal properties of code. By introducing scalable code transformations at the abstract syntax tree (AST) level and utilizing a differentiable feature space approximation, our approach enables robust, language-agnostic, and end-to-end trainable code watermarking. Experimental results demonstrate that SrcMarker achieves high accuracy and resilience against various code modifications, making it a promising solution for secure code ownership tracing.

1. Introduction

With the expansion of open-source software and the emergence of large language models, new ethical and security concerns regarding source code distribution have arisen. Issues such as copyright infringement, unauthorized redistribution, and malicious use of proprietary code are becoming increasingly prevalent. Tracking the ownership and authenticity of source code is thus of critical importance, and watermarking has emerged as a key technique to address these challenges.

However, few practical solutions exist for embedding watermarks in source code. Unlike general text or compiled binaries, source code is characterized by two distinct information channels: the *natural* channel, used by developers for understanding and maintenance, and the *formal* channel, which ensures that code can be processed and executed by compilers and tools. This duality adds complexity and subtlety to the watermarking process. Traditional software watermarking methods typically operate on binaries, often at the cost of code readability, while text watermarking approaches may disrupt code syntax or semantics, risking loss of functionality.

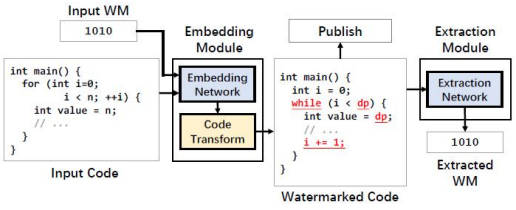

Figure 1.Overview of SrcMarker's embedding and extraction.

This paper introduces **SrcMarker**, a dual-channel source code watermarking framework designed to address these unique challenges.

2. Challenges

Effective source code watermarking faces several key challenges:

1. **Contextual Naturalness**: The naturalness of code is highly context-dependent, requiring the watermarking system to take contextual information into account. Rule-based methods are limited in this regard, often producing fixed or homogeneous patterns that are easily detectable.

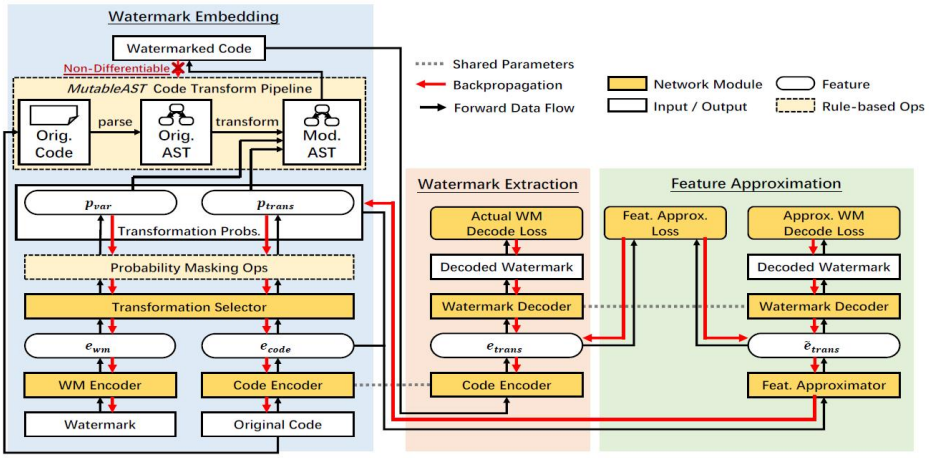

Figure 2. The architecture of SrcMarker. Note that the two "Code Encoder" blocks (in embedding and exraction modules respectively) refer to the sameneural network, and the same applies to the two "Watermark Decoder" blocks (in extraction and approximation modules).

2. **Formal Syntax Constraints**: Code must adhere strictly to syntactic rules. Any transformation must preserve syntactic validity and semantic correctness, making code transformation a delicate task.

3. **Discrete and Non-differentiable Nature**: Both source code and its transformations are inherently discrete and non-differentiable, which is fundamentally at odds with gradient-based, learnable designs. While heuristic or stochastic methods may partially circumvent this issue, they are unsuitable for end-to-end model training that requires differentiable operations.

3. Methodology

3.1 Overall Framework

Figure 2.The proposed **SrcMarker** system is designed to embed a watermark bit string into a given source code snippet. The system consists of three main modules: watermark embedding, watermark extraction, and feature approximation.

- **Watermark Embedding**: Inputs the watermark bit string and source code, outputs watermarked code.
- **Watermark Extraction**: Takes watermarked code as input and outputs the recovered watermark bits.
- **Feature Approximation**: Enables differentiable training over discrete code transformation operations.

3.2 Watermark Embedding Module

This module embeds the watermark bits into the source code through code transformations and variable substitutions while preserving code semantics and readability. Recognizing the dual-channel nature of code, transformations are applied at two levels:

- **Natural Channel**: Modifies variable names (e.g., `camelCase`, `PascalCase`, `snake_case`), affecting developer perception but not execution.
- **Formal Channel**: Alters program structures (e.g., converting between `for` and `while` loops or switching between `if-else` and `switch` constructs), impacting code structure without changing functionality.

The process involves encoding the input code and watermark bits into feature vectors, concatenating them, and passing the result to a transformation selector. This selector outputs probabilities for variable substitutions and code structure transformations, determining the final watermarking strategy. The process is designed to keep the two transformation types independent, preserving code correctness.

3.3 Watermark Extraction Module

The extraction module aims to recover the embedded watermark from watermarked code. The encoded transformed code is processed by a decoder that outputs the watermark bit string. Extraction accuracy depends on the robustness of both the embedding and extraction models.

3.4 Feature Approximation Module

A critical innovation of SrcMarker is its **feature space approximation**, which enables end-to-end training despite the discrete nature of code transformations. By employing **Gumbel-Softmax sampling** on one-hot vectors, the model achieves differentiability for backpropagation during training.

This module optimizes both the transformation selection strategy and the approximation of the true watermark extraction features. The model masks out impossible choices and randomly disables 50% of the remaining options to encourage exploration of diverse transformation strategies. Training is guided by loss functions measuring semantic similarity, code readability, and watermark extraction accuracy.

4. Contributions

The main contributions of this work are as follows:

1. **Dual-Channel Watermarking Framework**: We introduce a watermarking system that leverages both the natural and formal channels of source code, aligning with the intrinsic dual-channel nature of programming languages.

2. **AST-based Language-Agnostic Transformations**: By representing code as abstract syntax trees (ASTs), our approach performs program block synonym substitution, supporting language-agnostic and semantics-preserving code transformations at high speed.

3. **Differentiable Feature Space Approximation**: We propose a novel feature space approximation technique using Gumbel-Softmax, enabling end-to-end differentiable training for code watermarking. This technique can be extended to other code-related domains facing similar challenges, such as adversarial attack defense.

5. Conclusion

In this paper, we present SrcMarker, a dual-channel watermarking framework tailored to the unique properties of source code. By integrating AST-based program transformations and differentiable feature space approximation, SrcMarker achieves robust, language-agnostic, and trainable watermarking for source code. Our approach addresses the core challenges of code watermarking, including contextual naturalness, formal syntactic constraints, and the discrete nature of code transformations. The framework paves the way for more secure and reliable methods of source code ownership tracing, and future work may explore further applications in code security and intellectual property protection.